

# Sandbox Online Assessment System

## Post-Graduation Capstone Project Report

---

### Abstract

The **Sandbox Online Assessment System** is a full-stack online examination platform that allows organizations to create question banks, build timed assessments (MCQ and coding), assign them to candidates, and automatically evaluate and report results. This project was undertaken as a post-graduation capstone project to demonstrate practical, industry-relevant skills in full-stack development, microservice architecture, third-party API integration (code execution via Judge0), and secure authentication — built and delivered within a focused 12-day sprint.

---

## 1. Introduction

## 1.1 Motivation

Online assessment platforms are widely used by companies for technical hiring and by institutions for skill evaluation. Building one end-to-end demonstrates competence across the full stack: backend services, authentication/security, relational database design, a real code-execution pipeline, and a polished frontend — making it a strong, resume-ready capstone project after graduation.

## 1.2 Problem Statement

Organizations need a way to create standardized MCQ and coding assessments, assign them to candidates, and get automatically graded, trustworthy results without manual evaluation effort.

## 1.3 Objectives

- Design and implement a role-based platform for **Super Admin, HR (Organization Admin), and Candidate** roles.
- Build a question bank supporting both MCQ and coding questions.
- Implement a timed exam runtime with auto-save and auto-submit.
- Integrate a real code-execution engine (Judge0) for automatic coding-question grading.

- Implement basic integrity monitoring (browser tab-switch/fullscreen exit logging).
  - Deliver a working, demoable system within a **12-day implementation window**, suitable for a portfolio/capstone submission.
- 

## 2. Scope of the Project

To keep the project achievable within the 12-day academic timeline (rather than the full enterprise-scale original vision), scope was deliberately narrowed to what proves the core engineering skills without depending on unpredictable third-party services.

| Included in this Project                                                 | Deferred as Future Scope         |
|--------------------------------------------------------------------------|----------------------------------|
| JWT-based authentication, role-based access (Super Admin, HR, Candidate) | SSO / OAuth login                |
| Organization & HR account management                                     | Multi-branding per organization  |
| Candidate CRUD + basic CSV import                                        | Candidate self-onboarding portal |

| Included in this Project                                      | Deferred as Future Scope                              |
|---------------------------------------------------------------|-------------------------------------------------------|
| MCQ question bank (CRUD)                                      | Question tagging/analytics dashboard                  |
| Coding question bank with sample test cases                   | Difficulty-based auto-recommendation                  |
| Assessment builder (timer, passing marks, publish)            | Negative marking, scheduled windows                   |
| Candidate assignment + email notification                     | Bulk assignment via CSV                               |
| Exam runtime: timer, question palette, auto-save, auto-submit | Full crash-recovery/offline resilience                |
| Monaco code editor with Run/Submit                            | Multi-file coding projects                            |
| <b>Judge0-based automatic code grading</b>                    | LLM-based code-quality/complexity review              |
| Basic proctoring: tab-switch/blur/fullscreen-exit logging     | Camera-based AI face/phone detection, video recording |

| Included in this Project                              | Deferred as Future Scope                   |
|-------------------------------------------------------|--------------------------------------------|
| MCQ + coding result calculation, pass/fail            | AI-blended scoring                         |
| On-screen assessment report (scores, violation count) | PDF export, advanced analytics reports     |
| Single default subscription plan (no live payment)    | Razorpay live payment integration, billing |

This scoping choice mirrors real-world engineering practice — shipping a working core product first, and documenting integrations (like AI proctoring and payments) as designed-but-not-built future work is itself a demonstrable skill for a capstone report.

---

### 3. System Architecture

The system follows a **microservice-inspired architecture** (implementable as separate services or well-separated modules within a single deployable, depending on infrastructure available for the project):

1. **Authentication & Organization Service** — JWT auth, Spring Security (or equivalent),

organization/HR management.

2. **Assessment Engine Service** – candidates, question bank, assessment builder, assignment, exam runtime, results.
3. **Code Evaluation & Integrity Service** – Judge0 integration for code execution/grading, browser-event violation logging.

### Suggested Tech Stack

- **Backend:** Spring Boot (Java) or Node.js/Express
  - **Frontend:** Angular or React, with Monaco Editor for the coding workspace
  - **Database:** PostgreSQL / MySQL
  - **Code Execution:** Judge0 (hosted/cloud instance recommended for the project timeline)
  - **Auth:** JWT + BCrypt
  - **Email:** SMTP (e.g., SendGrid free tier) for assignment notifications
- 

## 4. Database Design (MVP Scope)

Core entities for this project's scope:

`users`, `roles`, `organizations`, `candidates`,  
`questions`, `mcq_options`, `coding_questions`,  
`test_cases`, `assessments`, `assessment_questions`,

assignments, submissions, coding\_submissions,  
results, violations

*(A full ER diagram with column-level detail should be produced as Appendix A of the final report before implementation begins.)*

---

## 5. Use Case Diagram

Three actors interact with the system: **Super Admin**, **HR (Organization Admin)**, and **Candidate**.

```
flowchart LR
    SA((Super Admin))
    HR((HR / Org Admin))
    CA((Candidate))

    subgraph System["Sandbox Online Assessment System"]
        UC1[Manage Organizations]
        UC2[Provision HR Accounts]
        UC3[Manage Candidates]
        UC4[Manage Question Bank]
        UC5[Build & Publish Assessment]
        UC6[Assign Assessment to Candidate]
        UC7[View HR Reports]
        UC8[Register / Login]
        UC9[Attempt MCQ Questions]
        UC10[Attempt Coding Questions]
        UC11[Submit Assessment]
        UC12[View Own Result]
    end
```

end

SA --> UC1

SA --> UC2

HR --> UC3

HR --> UC4

HR --> UC5

HR --> UC6

HR --> UC7

CA --> UC8

CA --> UC9

CA --> UC10

CA --> UC11

CA --> UC12

#### Use case summary table:

| Actor       | Use Case              | Description                                              |
|-------------|-----------------------|----------------------------------------------------------|
| Super Admin | Manage Organizations  | Create, update, activate/deactivate client organizations |
| Super Admin | Provision HR Accounts | Create the first HR/admin account for an organization    |
| HR          | Manage Candidates     | Add, edit, delete, search candidates; CSV import         |



| Actor     | Use Case                       | Description                                             |
|-----------|--------------------------------|---------------------------------------------------------|
| HR        | Manage Question Bank           | Create/edit MCQ and coding questions with test cases    |
| HR        | Build & Publish Assessment     | Configure timer/passing marks, add questions, publish   |
| HR        | Assign Assessment to Candidate | Select candidate(s), assign, trigger email notification |
| HR        | View HR Reports                | View scores, pass/fail status, and violation counts     |
| Candidate | Register / Login               | Create account or log in with JWT-based auth            |
| Candidate | Attempt MCQ Questions          | Answer MCQ questions within the timed exam              |
| Candidate | Attempt Coding Questions       | Write, run, and submit code in the Monaco editor        |

| Actor     | Use Case          | Description                                      |
|-----------|-------------------|--------------------------------------------------|
| Candidate | Submit Assessment | Manually submit or get auto-submitted at timeout |
| Candidate | View Own Result   | View score and pass/fail status after grading    |

## 6. System Flow Charts (End-to-End, with Valid & Invalid Paths)

The system has four distinct flows: a shared registration/login flow, the Super Admin flow, the HR flow, and the candidate exam flow. Each is shown separately for readability, with both valid (happy) paths and invalid/error paths at every decision point.

### 6.1 Registration & Login Flow (shared by all roles)

```
flowchart TD
    Start([User opens platform]) --> HasAccount{Has Account?}
    HasAccount -- No --> Register[Fill Registration Form]
    Register --> ValidateReg{Valid Details?}
```

```

\nEmail unique, password policy met}
    ValidateReg -- No --> RegError[Show
Error: Fix fields]
    RegError --> Register
    ValidateReg -- Yes -->
CreateUser[Create User Record\nHash
Password]
    CreateUser --> VerifyEmail[Send
Verification Email]
    VerifyEmail --> VerifyClick{Email
Verified?}
    VerifyClick -- No --> Pending[Account
Pending]
    Pending --> VerifyClick
    VerifyClick -- Yes --> Login

    HasAccount -- Yes --> Login[Enter Email
+ Password]
    Login --> AuthCheck{Credentials Valid?}
    AuthCheck -- No --> LoginError[Error:
Invalid credentials]
    LoginError --> Login
    AuthCheck -- Yes --> RoleCheck{Role?}
    RoleCheck -- Super Admin -->
AdminDash[Go to Admin Dashboard]
    RoleCheck -- HR --> HRDash[Go to HR
Dashboard]
    RoleCheck -- Candidate --> CandDash[Go
to Candidate Dashboard]

```

## 6.2 Super Admin Flow

```

flowchart TD
    A[Admin logs in] --> B[View Organization List]
    B --> C{Action?}
    C -- Create Org --> D[Enter Org Details]
    D --> E{Details Valid?\nName, domain unique}
    E -- No --> F[Show Error]
    F --> D
    E -- Yes --> G[Org Created]
    G --> H[Provision First HR Account]
    H --> I[Send HR Invite Email]
    I --> B
    C -- Activate/Deactivate --> J{Confirm Action?}
    J -- No --> B
    J -- Yes --> K[Update Org Status]
    K --> B
    C -- View Payments/Audit --> L[View Platform-wide Payment Log]
    L --> B

```

## 6.3 HR Flow

```

flowchart TD
    A[HR logs in] --> B{Action?}
    B -- Manage Candidates --> C[Add/Edit/Delete Candidate]

```

```
C --> D{Valid Input?}
D -- No --> E[Show Validation Error]
E --> C
D -- Yes --> F[Candidate Saved]
F --> B

B -- Manage Question Bank --> G[Add MCQ
or Coding Question]
G --> H{Valid?\nOptions/Test Cases
Present}
H -- No --> I[Show Error]
I --> G
H -- Yes --> J[Question Saved to Bank]
J --> B

B -- Build Assessment --> K[Create
Assessment\nAdd Questions, Set Timer &
Passing Marks]
K --> L{Config Valid?\nAt least 1
question, timer > 0}
L -- No --> M[Show Error]
M --> K
L -- Yes --> N[Publish Assessment]
N --> B

B -- Assign Assessment --> O[Select
Candidate + Assessment]
O --> P{Candidate Already Assigned?}
P -- Yes --> Q[Show Warning: Reassign?]
Q -- No --> B
Q -- Yes --> R[Update Assignment]
P -- No --> R[Create Assignment]
R --> S[Send Email Notification]
S --> B
```

```
B -- View Reports --> T[View Scores +  
Violations]  
T --> B
```

## 6.4 Candidate Exam Flow

The candidate exam flow below includes both the **valid (happy) path** and the **invalid/error paths** at every decision point: assignment check, exam-window expiry, permission denial, network/auto-save failure, code compile errors, and Judge0 service failure. A separate background branch shows integrity-violation monitoring (tab switch, window blur, fullscreen exit) running in parallel.

```
flowchart TD  
    Start([Candidate on Dashboard]) --> AssignCheck{Assessment\nAssigned to  
Candidate?}  
    AssignCheck -- No --> NoAccessError[Show Error:\nNo assessment  
assigned]  
    NoAccessError --> EndFail([End])  
  
    AssignCheck -- Yes --> WindowCheck{Within Start/End\nTime Window?}  
    WindowCheck -- No / Expired --> WindowError[Show Error:\nAssessment window  
closed]  
    WindowError --> EndFail
```

```

    WindowCheck -- Yes -->
Permissions[Request Camera & Fullscreen
Permission]
    Permissions --> PermCheck{Permission
Granted?}
    PermCheck -- Denied --> PermError[Show
Error:\nPermission required to start exam]
    PermError --> Permissions

    PermCheck -- Granted -->
StartExam[Start Exam - Timer Begins]
    StartExam --> QuestionLoop[Load Next
Question]

    QuestionLoop --> QType{Question Type?}

    QType -- MCQ --> MCQAnswer[Candidate
Selects Option]
    MCQAnswer --> MCQSave{Auto-Save
Successful?}
    MCQSave -- No / Network Error -->
MCQRetry[Retry Save in Background\nShow
\n"Saving...\n" warning]
    MCQRetry --> MCQSave
    MCQSave -- Yes --> MoreQ

    QType -- Coding --> CodeWrite[Candidate
Writes Code in Monaco Editor]
    CodeWrite --> RunCode[Click Run Code]
    RunCode --> Judge0Run[Judge0 Executes
vs Sample Test Cases]
    Judge0Run --> RunResult{Compiles
&\nRuns OK?}
    RunResult -- No --> CompileError[Show

```

```
Compilation/\nRuntime Error to Candidate]
  CompileError --> CodeWrite
  RunResult -- Yes --> ShowOutput[Show
Output to Candidate]
  ShowOutput --> SubmitDecision{Candidate
Submits Code?}
  SubmitDecision -- Not Yet --> CodeWrite
  SubmitDecision -- Yes -->
SubmitCode[Click Submit Code]
  SubmitCode --> Judge0Hidden[Judge0
Executes vs Hidden Test Cases]
  Judge0Hidden --> HiddenResult{Judge0
Service\nResponded?}
  HiddenResult -- Timeout/Error -->
Judge0Fail[Log Failure - Flag for\nManual
Re-evaluation]
  Judge0Fail --> MoreQ
  HiddenResult -- Success -->
CodeScore[Store Pass/Fail per Test Case +
Score]
  CodeScore --> MoreQ[More Questions?]

  MoreQ --> TimerCheck{Timer Expired?}
  TimerCheck -- Yes --> ForceSubmit[Auto-
Submit Exam]
  TimerCheck -- No -->
MoreQDecision{Questions Remaining?}
  MoreQDecision -- Yes --> QuestionLoop
  MoreQDecision -- No -->
ManualSubmit{Candidate Clicks\nSubmit
Exam?}
  ManualSubmit -- No --> QuestionLoop
  ManualSubmit -- Yes -->
ConfirmSubmit[Confirm Submission Dialog]
```



```

    ConfirmSubmit --> FinalSubmit[Submit
Exam]

    ForceSubmit --> Grading
    FinalSubmit --> Grading[Result Engine
Calculates\nMCQ Score + Coding Score]

    Grading --> PassCheck{Score >= Passing
Marks?}
    PassCheck -- Yes --> Pass[Status: PASS]
    PassCheck -- No --> Fail[Status: FAIL]

    Pass --> ResultView[Result Shown to
Candidate\n+ Available to HR]
    Fail --> ResultView
    ResultView --> EndSuccess([End])

    StartExam --> Monitor[Background:
Monitor Tab Switch,\nWindow Blur,
Fullscreen Exit]
    Monitor --> ViolationCheck{Violation
Detected?}
    ViolationCheck -- Yes -->
LogViolation[Log Violation Event\nwith
Timestamp]
    LogViolation --> Monitor
    ViolationCheck -- No --> Monitor
    LogViolation --> ResultView

```

**Flow narrative – valid path:** user registers/logs in successfully and lands on the correct role dashboard  
→ (Admin) creates org and provisions HR → (HR) adds candidates, builds a question bank, publishes an

assessment, assigns it, and later reviews reports → (Candidate) assessment is assigned and within its time window, permissions granted, exam starts, each MCQ auto-saves successfully, each coding submission compiles and runs correctly, all questions answered, candidate submits (or timer expires and it auto-submits), result is calculated and shown to both candidate and HR.

**Flow narrative – invalid/error paths handled:**

duplicate email at registration, unverified email, wrong login credentials, duplicate org name/domain, invalid question bank entries (missing options/test cases), invalid assessment config (no questions/zero timer), re-assigning an already-assigned candidate, candidate with no assessment assigned, entering after the assessment window has closed, camera/fullscreen permission denial, auto-save network failures (retried in background), code that fails to compile/run, and Judge0 service timeout/failure (flagged for manual re-evaluation rather than silently failing). Integrity violations (tab switch, blur, fullscreen exit) are logged in a background branch that runs independently of the main exam flow and feeds into the HR-visible result.

---

## 7. Implementation Plan (12-Day Timeline)

Framed for a capstone project — whether built solo or with a small student group, each "day" below represents a milestone rather than a fixed team assignment.

| Day | Milestone                                                                                               |
|-----|---------------------------------------------------------------------------------------------------------|
| 1   | Requirement finalization, ER diagram, API contract sketch, project scaffolding (repo, environments)     |
| 2   | Auth module: register/login/JWT; Candidate CRUD; DB schema for question bank                            |
| 3   | Organization/HR management; MCQ CRUD complete; Judge0 integration spike (one working sample submission) |
| 4   | Assignment module + email notification; Coding question CRUD; Monaco editor shell                       |
| 5   | Assessment builder (create/add questions/publish); connect Auth/Candidate/Question Bank screens to APIs |
| 6   | Exam runtime v1: timer, palette, MCQ answer-save wired end-to-end                                       |

| Day | Milestone                                                                                                                                 |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | Coding runtime: Monaco Run/Submit wired to Judge0; violation-event listeners (tab switch/blur/fullscreen)                                 |
| 8   | Auto-save + auto-submit-on-timeout; result calculation engine (MCQ + coding)                                                              |
| 9   | <b>End-to-end demo run:</b> HR creates assessment → assigns candidate → candidate completes exam → result generated. Freeze new features. |
| 10  | Bug fixing based on Day 9 demo; RBAC/validation gap fixes                                                                                 |
| 11  | Basic report screen; email template polish; documentation write-up begins                                                                 |
| 12  | Final testing, deployment to a free-tier host (for demo purposes), project report finalization, presentation prep                         |

---

## 8. Learning Outcomes

Completing this project demonstrates the following post-graduation-ready skills:

- Designing and implementing role-based access control and JWT authentication from scratch.
  - Relational database design for a multi-tenant system.
  - Integrating a real third-party code-execution API (Judge0) into an application workflow.
  - Building a timed, stateful exam-runtime UI with auto-save/auto-submit logic.
  - Scoping a large system down to a shippable MVP under a real deadline — a core software engineering skill, not just a coding one.
- 

## 9. Future Scope

The following were identified during planning as valuable extensions beyond the 12-day academic timeline, and are recommended as follow-up work or a "v2" of the project:

- AI-based proctoring: camera snapshot analysis for face/phone detection, event-triggered video recording.
- LLM-based code quality, complexity, and optimization feedback.
- Live payment gateway integration (Razorpay) with subscription billing.

- PDF report generation and advanced analytics dashboards.
  - Candidate self-registration and bulk CSV assignment.
- 

## **10. Conclusion**

The Sandbox Online Assessment System, scoped to its 12-day MVP, is a realistic and achievable post-graduation capstone project that still touches every layer of a modern SaaS application — authentication, relational data modeling, third-party API integration, and a non-trivial stateful frontend (timed exam runtime with a code editor). Framing the AI-proctoring and payment features as documented future scope, rather than attempting to build them under time pressure, keeps the delivered project both complete and credible for academic submission or a professional portfolio.